

后端 / 全栈开发 2 天技术测验

目标：构建一个简化版的，类 Polymarket.com 的预测 (Forecasting) 平台核心逻辑，重点考察账户余额管理、幂等性处理、状态机控制及账本对账机制。

技术栈建议

- 后端: Node.js (TypeScript)
- 前端: Next.js
- 数据库: SQLite + Prisma (推荐)
- 部署: Vercel (可选)

一、基础功能说明

1. 用户系统 (User)

系统需预置静态用户数据（数据库迁移/种子数据预置，不允许动态生成）。

- 核心字段: id, username, balance (初始余额), createdAt

2. 充值接口 (Deposit)

- 接口: POST /api/users/:id/deposit
- Header: Idempotency-Key: <string>
- Body: { "amount": number }
- 要求:
- 成功充值后增加余额。
- 必须支持幂等: 相同 Idempotency-Key 重复请求只能生效一次。
- 若相同 Key 但金额不一致, 应返回 409 Conflict。

3. 下注接口 (Bet)

- 接口: POST /api/bets
- Header: Idempotency-Key: <string>
- Body: { "userId": number, "gameId": string, "amount": number }
- 要求:
- 严禁负余额: 余额不足时必须报错。
- 扣减余额并创建 Bet 记录。
- 必须支持幂等处理。

二、状态机逻辑 (State Machine)

下注状态流转

- 状态枚举: PLACED, SETTLED, CANCELLED
- 流转规则:
- PLACED → SETTLED (结算成功)
- PLACED → CANCELLED (取消/退款)
- SETTLED / CANCELLED 为终态, 不可再次变更, 不允许重复结算。

4. 结算接口 (Settle)

- 接口: POST /api/bets/:id/settle
- Body: { "result": "WIN" | "LOSE" }
- 规则:
- WIN: 增加奖金 (原路返还并加上盈利)。
- LOSE: 无余额返还。

5. 取消接口 (Cancel)

- 接口: POST /api/bets/:id/cancel
- 规则: 仅允许在 PLACED 状态下执行, 必须执行退款并将状态置为 CANCELLED。

三、账本模型 (Ledger Model)

必须采用 追加式账本 (Append-only Ledger) 设计, 禁止直接修改历史账务记录。

Ledger 类型	触发场景
DEPOSIT	用户充值成功
BET_DEBIT	用户下单扣费
BET_CREDIT	结算获胜 (WIN) 发奖
BET_REFUND	订单取消 (CANCELLED) 退款

核心原则: 不允许直接在业务代码中随意修改 `User.balance` 字段。余额应保证与账本记录的加总 (Sum) 逻辑一致。

四、对账接口 (Admin Reconciliation)

- 接口: `GET /api/admin/reconcile?userId=...`
- 功能: 检查账务一致性, 返回以下信息:
 - 当前数据库记录余额。
 - 由账本记录推导出的计算余额。
 - 各状态订单统计。
 - 异常发现: 是否存在缺少扣款记录、重复结算、退款缺失等异常。

五、自动化测试要求

必须包含至少 6 个核心测试用例 (推荐使用 Jest 或 Vitest) :

- 充值成功后余额正确增加。
- 充值幂等性验证 (多次请求, 一次生效)。
- 余额不足时, 下注应当失败。
- 下注操作的幂等性验证。
- 结算为 WIN 时, 余额正确增加。
- 已结算订单不允许重复结算。

六、提交与评估

技术要求

- 分层架构: 职责分离 (Service/Logic 不应写在 Controller 中)。
- 一致性保证: 在处理余额与账本时, 必须合理使用 **数据库事务 (Transaction)**。
- 鲁棒性: 包含基本的错误处理机制。

提交内容

- GitHub 仓库地址。
- README: 包含运行步骤、测试命令、API 说明及 (可选的) 在线预览地址。

评分重点

- 幂等性与状态机的实现严谨度。
- 账本一致性与事务处理。
- 边界情况处理 (并发、异常状态流转)。
- 代码的可读性与测试覆盖度。